# DEVSTUDIO

Project and viva guide

A beginner-friendly guide to how the project is organized, what its main code does, and how to explain the technologies used.

| Project area | What it demonstrates |
| --- | --- |
| Resume | React state, controlled inputs, live preview, print styles, and a basic keyword comparison |
| Portfolio | Project add, edit, delete, and public GitHub repository fetching |
| Forms | Editable form fields, input types, browser validation, and response display |
| Dashboard | Navigation, shared project state, templates, theme controls, and reset to sample data |
| Browser storage | localStorage caches fetched job listings for the resume checker |

Prepared from the current DevStudio source files. Intended for a student presentation and viva.

# 1. What DevStudio does

DevStudio is a single-page React application for building a personal developer presence. It brings together a dashboard, resume editor, project portfolio, and response-collecting contact form. Navigation switches the visible section without loading a new page.

The main data is held in App.jsx. App passes the information and update functions to each section as props. This lets the resume and portfolio use the same projects, and lets the dashboard show current project and response counts.

## Simple viva explanation

> *"DevStudio is a browser-based personal branding workspace. It uses React components for separate sections, state to update the screen, forms for user input, and APIs to load public GitHub repositories and job listings."*

## Project file map

| File | Purpose |
|---|---|
| src/main.jsx | Starts the React application and renders App. |
| src/App.jsx | Keeps shared state, handles common actions, and chooses which section to show. |
| src/components/Dashboard.jsx | Displays what DevStudio does, the workflow, project and response counts, themes, and reset action. |
| src/components/Resume.jsx | Edits resume details, selects projects, shows a live preview, and prints. |
| src/components/AtsChecker.jsx | Loads live job posts, compares a small set of keywords, and caches job data. |
| src/components/Portfolio.jsx | Displays projects and supports inline add/edit/delete plus GitHub repository loading. |
| src/components/Forms.jsx | Edits form fields and types, displays the form preview, and lists responses. |
| CSS files | Each component has its own styles; style.css contains shared app layout and navigation. |

# 2. React, JSX, and shared state

JSX lets the JavaScript files describe the HTML-like page structure. Components are functions that return JSX. App.jsx imports each page component and passes it the data it needs.

## State with useState

useState stores a value that can change while the app is open. When a setter changes the value, React renders the affected part of the page again.

```
const [active, setActive] = useState('Overview');
const [projects, setProjects] = useState(initialProjects);
```

In DevStudio, active decides which section is visible. projects stores the current portfolio. setProjects updates it after adding, editing, deleting, or importing a project.

## Props and component composition

Props are values and functions passed from a parent component to a child. App passes profile, projects, and event handlers to Resume, Portfolio, and Forms. Smaller components such as ProjectCard and ProjectForm are composed inside Portfolio.

```
<Resume
  profile={profile}
  setField={setProfileField}
  projects={projects}
/>
```

## Events and conditional rendering

An event handler runs after an action such as a click or form submission. Conditional rendering uses a condition to decide which JSX to show. App checks active and renders only the selected screen.

```
{active === 'Resume' && (
  <Resume profile={profile} projects={projects} />
)}
```

## Array methods used

| Method | Example in DevStudio | Why it is used |
|---|---|---|
| map | Render project cards, response rows, and form fields. | Turns each data item into a JSX element. |
| filter | Remove a project or a form field. | Keeps items that pass a condition. |
| some | Check whether a GitHub repository is already added. | Returns true when at least one item matches. |
| forEach | Find the next available form field number. | Runs a small action for each item. |
| split / trim | Convert comma-separated skills into tags. | Breaks text into clean values. |

# 3. Resume editor and preview

Resume.jsx receives the profile object and project list through props. Text inputs are controlled: their value comes from React state, and onChange sends the new value back to App.jsx. The preview reads the same state, so it changes as the person types.

```
<input
  value={profile.name}
  onChange={(event) => setField('name', event.target.value)}
/>
```

The setProfileField function updates one property while keeping the rest of the profile object. The spread operator copies the old fields; the computed property name updates the requested field.

```
setProfile((currentProfile) => ({
  ...currentProfile,
  [field]: value,
}));
```

Project checkboxes let the user choose which portfolio projects appear in the resume. Print or save as PDF calls the browser print dialog. CSS print rules can hide or simplify interface elements for printing.

## Resume keyword checker

The checker fetches job postings from Arbeitnow. It filters the returned jobs for frontend-related titles or tags. A list of common development keywords is searched in the selected job description and in resume text. It then displays the found keywords, missing keywords, and a simple percentage.

```
const response = await fetch(jobApiUrl);
const result = await response.json();
const frontendJobs = (result.data || []).filter((job) => {
  const title = job.title.toLowerCase();
  return title.includes('frontend') || title.includes('react');
});
```

This is a beginner keyword comparison, not an official ATS evaluation. The job data comes from a live API; the percentage calculation runs in the browser.

# 4. Portfolio and GitHub API

The portfolio stores each project as an object with a title, type, description, and tags. An inline form uses FormData to read submitted values. The same form is used to add a project or edit an existing one. Delete removes it from the shared projects array.

The GitHub area asks for a username, then uses fetch to request up to ten public repositories. encodeURIComponent makes the username safe to place in the URL. The code checks the response status, converts JSON into JavaScript data, and shows loading or error messages.

```
const response = await fetch(
  `https://api.github.com/users/${encodeURIComponent(username)}/repos?per_page=10`,
);
const repositoryList = await response.json();
setRepositories(repositoryList);
```

A selected public repository can be added to the portfolio. The app checks for duplicates before adding it. This is a real API request, but it only reads public repository information.

## Forms and validation

The Forms section stores its editable field definitions in React state. Adding a field creates another object in the fields array. Editing a label or answer type updates that object. Deleting a field filters it out of the array. The preview renders the same field list, so changes appear immediately.

The selected answer type becomes the input type, such as text, email, number, or date. The browser then provides basic input behavior and validation. The form submission stores name, email, message, and date in React state for the current visit. It does not save responses to a backend or database.

# 5. Local storage and refresh behavior

DevStudio uses localStorage only to cache up to 20 frontend job listings fetched by the resume checker. The browser keeps this small cache for the same website even after a refresh or when the tab is closed.

## Saving to localStorage

```
localStorage.setItem(
  'devstudio-saved-jobs',
  JSON.stringify(frontendJobs),
);
```

localStorage stores text, so JSON.stringify converts the job array into a text string before saving it.

## Reading the cached values

```
const savedJobs = localStorage.getItem('devstudio-saved-jobs');
const parsedJobs = JSON.parse(savedJobs);
```

getItem reads the text value. JSON.parse converts it back into an array of job objects. The app uses a try/catch in case storage is unavailable or the saved text is not valid JSON.

## What persists and what resets

| Data | After refresh | Reason |
|---|---|---|
| Fetched job listings | Can be restored | Cached in localStorage by the resume checker. |
| Resume edits | Reset to samples | Stored only in React state. |
| Added or deleted projects | Reset to samples | Stored only in React state. |
| Form responses | Reset to empty | Stored only in React state. |
| Form fields and theme | Reset to defaults | Stored only in React state. |

To see the cache in the browser, first open Resume and choose Load live jobs. Then open Developer Tools and inspect Application (Chrome) or Storage (Firefox) for the current site. Find the key devstudio-saved-jobs. The application does not display the browser storage panel itself.

This is a simple localStorage example for the course topic. A production app would also consider when cached job data should expire.

# 6. HTML and CSS topics

The JSX uses semantic HTML elements such as main, aside, nav, header, section, article, form, label, and button. Labels describe form controls, while aria-label and aria-live provide additional context to assistive technology in some places.

Separate CSS files keep styles close to their components. The project uses the box model through padding, margins, borders, and element sizing. Flexbox arranges items in a row and aligns them. Grid creates multi-column layouts for the resume, dashboard, and forms.

Responsive CSS uses media queries to change layouts on smaller screens. For example, the resume layout changes from two columns to one column. This makes the interface more usable on narrow screens.

## Basic styles used

| CSS topic | How DevStudio uses it |
| --- | --- |
| Box model | Padding, border, and margin create spacing around panels, inputs, and cards. |
| Flexbox | Aligns headings, controls, navigation items, and card contents. |
| Grid | Creates columns for editor/preview and dashboard cards. |
| Media queries | Switches multi-column layouts to a single column on smaller screens. |
| Print styles | Adjust resume presentation when using the browser print dialog. |

## How to describe CSS in the viva

> *"CSS controls layout and appearance. Flexbox aligns items, Grid makes columns, and media queries adjust those layouts for smaller screens. The resume also has print styles so it can be saved from the browser as a PDF."*

# 7. Syllabus topic coverage

This project demonstrates several syllabus topics directly. Other topics are not currently included, or appear only at a basic level.

| Syllabus topic | Coverage in DevStudio |
| --- | --- |
| HTML structure and semantic tags | Used in the app layout and component markup. |
| Accessibility basics | Labels and some ARIA attributes are present; accessibility is not explored in depth. |
| CSS fundamentals and box model | Used across all component styles. |
| Flexbox, Grid, responsive design, media queries | Used in page and card layouts. |
| Variables, functions, arrays, objects, events | Used throughout React components. |
| ES6 modules and imports/exports | Each component and CSS file is imported as a module. |
| Promises and async/await | API functions use async/await; promises are used through fetch. |
| DOM manipulation | React updates the UI declaratively; direct manual DOM editing is not a project pattern. |
| Browser storage and JSON | localStorage caches job results using JSON.stringify and JSON.parse. |
| React components, props, state, lists, conditional rendering | Core structure of the app. |
| React Router and route parameters | Not used; app sections switch with React state. |
| Context API / Redux / Zustand | Not used; shared state is passed as props from App. |
| useEffect, useRef, useMemo, useCallback, custom hooks | Not used in this project. |
| Form handling and basic validation | Dynamic form controls, FormData, input types, and browser-required validation. |
| API loading and error messages | Basic loading and error states are used for GitHub and job API requests. |
| Pagination, infinite scroll, debounce, retry | Not used. |
| Backend and persistent form database | Not used; app data is local to the browser session except job cache. |

# 8. Viva questions and short answers

### What is React state?

State is data that can change. React renders the page again when a state setter updates it.

### What are props?

Props are values or functions passed from a parent component to a child component.

### What is a controlled input?

Its displayed value comes from React state, and its onChange event updates that state.

### What does fetch do?

fetch sends an HTTP request to an API and returns a promise with the response.

### Why use async and await?

They let the code wait for a promise result using a readable step-by-step style.

### What does JSON do here?

JSON is the data format returned by APIs and used to convert the job cache to and from text.

### Is the resume checker an actual ATS?

No. It compares a fixed list of common keywords against a live job posting and resume text. It is not an employer ATS or official score.

### Where is localStorage used?

AtsChecker.jsx saves fetched job listings under devstudio-saved-jobs and reads them after refresh.

### Why do resume edits reset after refresh?

Profile and project data are held in React state and are not written to localStorage. Only fetched job listings are cached.

### Why not React Router?

The app is a small single-page workspace. Its four sections are selected with state, so a routing library was not needed for this version.

### How can I save the resume as a PDF?

Choose Print or save as PDF to open the browser print dialog, then choose Save as PDF.

## Quick demonstration order

| Step | Action to show |
|---|---|
| 1 | Open the dashboard and explain the four sections. |
| 2 | Edit a resume field and show the preview update. |
| 3 | Choose which projects to show, then print or save the resume. |
| 4 | Load live jobs, choose a role, and show matching and missing keywords. |
| 5 | Inspect the devstudio-saved-jobs key in browser Developer Tools after loading jobs. |
| 6 | Add or edit a project, then show the public GitHub repository loader. |
| 7 | Add and delete a form field, change its answer type, and submit a response. |

Reference: Arbeitnow public job API - https://www.arbeitnow.com/api/job-board-api

The API connection needs internet access. Job listings may change over time.